

CHULETA — FICHEROS EN JAVA

1. **File** — Fichero o directorio

 `java.io.File`

¿Para qué sirve?

Representa la ruta de un fichero o directorio. **No lee ni escribe su contenido.**

```
File f = new File("datos/alumnos.txt");
```

Métodos importantes

Método	¿Para qué sirve?
<code>exists()</code>	Comprueba si existe
<code>isFile()</code>	Comprueba si es un fichero
<code>isDirectory()</code>	Comprueba si es un directorio
<code>getName()</code>	Obtiene el nombre
<code>getPath()</code>	Obtiene la ruta
<code>getAbsolutePath()</code>	Obtiene la ruta absoluta
<code>length()</code>	Tamaño en bytes
<code>mkdir()</code>	Crea un directorio
<code>mkdirs()</code>	Crea directorios necesarios
<code>createNewFile()</code>	Crea un fichero vacío
<code>delete()</code>	Elimina fichero/directorio
<code>listFiles()</code>	Obtiene los ficheros de un directorio

Ejemplo

```
File carpeta = new File("datos");
```

```
if (carpeta.exists() && carpeta.isDirectory()) {  
    for (File f : carpeta.listFiles()) {
```

```
        System.out.println(f.getName());
    }
}
```

2. **FileReader** — Leer texto

 `java.io.FileReader`

¿Para qué sirve?

Leer caracteres de un fichero de texto.

```
FileReader fr = new FileReader("alumnos.txt");
```

Método importante

Método	¿Para qué sirve?
--------	------------------

<code>read()</code>	Lee un carácter
---------------------	-----------------

<code>close()</code>	Cierra el fichero
----------------------	-------------------

También puede utilizarse `read(char[])`.

⚠ En la práctica, para leer texto línea a línea suele ser más cómodo utilizarlo junto con `BufferedReader`.

3. **BufferedReader** — Leer texto línea a línea

 `java.io.BufferedReader`

¿Para qué sirve?

Leer texto de forma cómoda y eficiente, especialmente **líneas completas**.

```
BufferedReader br =
    new BufferedReader(new FileReader("alumnos.txt"));
```

Método estrella

`readLine()`

Lee una línea.

Cuando llega al final:

`null`

Ejemplo

`String linea;`

```
while ((linea = br.readLine()) != null) {  
    System.out.println(linea);  
}
```

Otros métodos:

Método	¿Para qué sirve?
<code>readLine()</code>	Lee una línea
<code>read()</code>	Lee un carácter
<code>close()</code>	Cierra el recurso

4. `FileWriter` — Escribir texto

 `java.io.FileWriter`

¿Para qué sirve?

Escribir caracteres en un fichero de texto.

```
FileWriter fw = new FileWriter("alumnos.txt");
```

Métodos importantes

Método	¿Para qué sirve?
--------	------------------

<code>write()</code>	Escribe texto
---------------------------	---------------

<code>flush()</code>	Fuerza la escritura
---------------------------	---------------------

```
close(  Cierra el fichero
)
```

Sobrescribir

```
new FileWriter("alumnos.txt")
```

Añadir al final

```
new FileWriter("alumnos.txt", true)
```

Muy importante recordar el **true**.

5. **BufferedWriter** — Escribir texto por líneas

 `java.io.BufferedWriter`

¿Para qué sirve?

Escribir texto de forma eficiente y permite escribir líneas fácilmente.

```
BufferedWriter bw =
    new BufferedWriter(new FileWriter("alumnos.txt"));
```

Métodos importantes

Método	¿Para qué sirve?
<code>write()</code>	Escribe texto
<code>newLine()</code>	Salto de línea
<code>flush()</code>	Fuerza la escritura
<code>close()</code>	Cierra el recurso

Ejemplo

```
bw.write("Juan;20;DAW");
bw.newLine();
bw.write("Ana;21;DAM");
```

6. Properties — Ficheros

.properties

 `java.util.Properties`

¿Para qué sirve?

Guardar y leer **configuración** en formato:

```
usuario=admin  
idioma=es  
puerto=8080
```

Métodos importantes

Método	¿Para qué sirve?
<code>setProperty()</code>	Añade/modifica una propiedad
<code>getProperty()</code>	Obtiene una propiedad
<code>load()</code>	Carga propiedades desde un fichero
<code>store()</code>	Guarda propiedades en un fichero
<code>containsKey()</code>	Comprueba si existe una clave
<code>remove()</code>	Elimina una propiedad

Leer

```
Properties p = new Properties();  
  
p.load(new FileReader("config.properties"));  
  
String usuario = p.getProperty("usuario");
```

Escribir

```
Properties p = new Properties();  
  
p.setProperty("usuario", "admin");  
p.setProperty("idioma", "es");  
  
p.store(new FileWriter("config.properties"),
```

"Configuración");

7. `FileInputStream` — Leer bytes

 `java.io.FileInputStream`

¿Para qué sirve?

Leer información **binaria**, byte a byte.

```
FileInputStream fis =  
    new FileInputStream("datos.dat");
```

Métodos importantes

Método	¿Para qué sirve?
<code>read()</code>	Lee un byte
<code>read(byte[])</code>	Lee varios bytes
<code>available()</code>	Bytes disponibles
<code>close()</code>	Cierra el recurso

Se utiliza, por ejemplo, para:

imágenes
PDF
audio
vídeo
ficheros .dat

8. `FileOutputStream` — Escribir bytes

 `java.io.FileOutputStream`

¿Para qué sirve?

Escribir información binaria.

```
FileOutputStream fos =
```

```
new FileOutputStream("datos.dat");
```

Métodos importantes

Método	¿Para qué sirve?
<code>write()</code>	Escribe un byte
<code>write(byte[])</code>	Escribe varios bytes
<code>flush()</code>	Fuerza la escritura
<code>close()</code>	Cierra el recurso

Para añadir en lugar de sobrescribir:

```
new FileOutputStream("datos.dat", true)
```

9. `DataInputStream` — Leer tipos primitivos

 `java.io.DataInputStream`

¿Para qué sirve?

Leer tipos de datos Java desde un fichero binario.

Normalmente se coloca encima de un `FileInputStream`:

```
DataInputStream dis =  
    new DataInputStream(  
        new FileInputStream("datos.dat"));
```

Métodos importantes

Método	Lee
<code>readInt()</code>	<code>int</code>
<code>readDouble()</code>	<code>double</code>
<code>readFloat()</code>	<code>float</code>
<code>readLong()</code>	<code>long</code>
<code>readBoolean()</code>	<code>boolean</code>

<code>readChar()</code>	<code>char</code>
<code>readUTF()</code>	<code>String</code>
<code>readByte()</code>	<code>byte</code>
<code>readShort()</code>	<code>short</code>

Ejemplo:

```
int edad = dis.readInt();
double nota = dis.readDouble();
String nombre = dis.readUTF();
```

10. `DataOutputStream` — Escribir tipos primitivos

 `java.io.DataOutputStream`

¿Para qué sirve?

Escribir tipos de datos Java en formato binario.

```
DataOutputStream dos =
    new DataOutputStream(
        new FileOutputStream("datos.dat"));
```

Métodos importantes

Método	Escribe
<code>writeInt()</code>	<code>int</code>
<code>writeDouble()</code>	<code>double</code>
<code>writeFloat()</code>	<code>float</code>
<code>writeLong()</code>	<code>long</code>
<code>writeBoolean()</code>	<code>boolean</code>
<code>writeChar()</code>	<code>char</code>
<code>writeUTF()</code>	<code>String</code>
<code>writeByte()</code>	<code>byte</code>

`writeShort()` `short`

⚠ Regla de oro

Lo que escribes:

```
dos.writeUTF(nombre);  
dos.writeInt(edad);  
dos.writeDouble(nota);
```

debes leerlo:

```
dis.readUTF();  
dis.readInt();  
dis.readDouble();
```

En el mismo orden y con el mismo tipo.

11. `ObjectOutputStream` — Guardar objetos

📦 `java.io.ObjectOutputStream`

¿Para qué sirve?

Guardar objetos Java completos mediante **serialización**.

La clase debe implementar:

`implements Serializable`

Ejemplo:

```
ObjectOutputStream oos =  
    new ObjectOutputStream(  
        new FileOutputStream("alumnos.dat"));
```

```
oos.writeObject(alumno);
```

Método fundamental:

```
writeObject()
```

12. **ObjectInputStream** — Recuperar objetos

 `java.io.ObjectInputStream`

¿Para qué sirve?

Leer objetos que previamente hemos serializado.

```
ObjectInputStream ois =  
    new ObjectInputStream(  
        new FileInputStream("alumnos.dat"));
```

```
Alumno alumno = (Alumno) ois.readObject();
```

Método fundamental:

```
readObject()
```

Puede aparecer:

`ClassNotFoundException`

13. **IOException** — La excepción estrella

 `java.io.IOException`

Es la excepción general relacionada con operaciones de entrada/salida.

Ejemplo:

```
try {  
    // operaciones con ficheros  
} catch (IOException e) {  
    System.out.println("Error de entrada/salida");  
}
```

Muchas clases de ficheros pueden producirla.

14. FileNotFoundException

 `java.io.FileNotFoundException`

Se produce cuando un fichero **no puede abrirse**.

Por ejemplo:

```
try {
    FileReader fr =
        new FileReader("inexistente.txt");

} catch (FileNotFoundException e) {
    System.out.println("El fichero no existe");
}
```

Es una excepción derivada de `IOException`.

`IOException`



`FileNotFoundException`

15. EOFException

 `java.io.EOFException`

EOF = End Of File

Se produce cuando intentamos leer más datos de los que realmente hay en determinados flujos.

Es especialmente interesante con:

`DataInputStream`

Ejemplo conceptual:

```
try {
    while (true) {
        int numero = dis.readInt();
        System.out.println(numero);
    }
} catch (EOFException e) {
    System.out.println("Fin del fichero");
}
```

16. try-with-resources

La forma recomendada actualmente para trabajar con recursos que deben cerrarse.

En lugar de:

```
BufferedReader br =  
    new BufferedReader(new FileReader("datos.txt"));
```

...

```
br.close();
```

podemos hacer:

```
try (BufferedReader br =  
    new BufferedReader(  
        new FileReader("datos.txt"))) {  
  
    String linea;  
  
    while ((linea = br.readLine()) != null) {  
        System.out.println(linea);  
    }  
  
} catch (IOException e) {  
    System.out.println("Error");  
}
```

Ventaja principal

El recurso se **cierra automáticamente**.



CHULETA ULTRARRÁPIDA

Si quieres que los alumnos tengan una sola tabla para consultar:

Clase

¿Para qué?

Métodos clave

<code>File</code>	Fichero/directorio	<code>exists()</code> , <code>isFile()</code> , <code>isDirectory()</code> , <code>length()</code> , <code>delete()</code> , <code>listFiles()</code>
<code>FileReader</code>	Leer texto	<code>read()</code> , <code>close()</code>
<code>FileWriter</code>	Escribir texto	<code>write()</code> , <code>close()</code>
<code>BufferedReader</code>	Leer texto/líneas	<code>readLine()</code> , <code>read()</code> , <code>close()</code>
<code>BufferedWriter</code>	Escribir texto	<code>write()</code> , <code>newLine()</code> , <code>close()</code>
<code>Properties</code>	Configuración	<code>getProperty()</code> , <code>setProperty()</code> , <code>load()</code> , <code>store()</code>
<code>FileInputStream</code>	Leer bytes	<code>read()</code> , <code>close()</code>
<code>FileOutputStream</code>	Escribir bytes	<code>write()</code> , <code>close()</code>
<code>DataInputStream</code>	Leer tipos Java	<code>readInt()</code> , <code>readDouble()</code> , <code>readUTF()</code> , <code>readBoolean()</code>
<code>DataOutputStream</code>	Escribir tipos Java	<code>writeInt()</code> , <code>writeDouble()</code> , <code>writeUTF()</code> , <code>writeBoolean()</code>
<code>ObjectInputStream</code>	Leer objetos	<code>readObject()</code>
<code>ObjectOutputStream</code>	Guardar objetos	<code>writeObject()</code>

Y la regla mental

¿Quiero información del fichero?



`File`

¿Quiero texto?



`BufferedReader` / `BufferedWriter`

¿Quiero configuración?



`Properties`

¿Quiero bytes?



`FileInputStream` / `FileOutputStream`

¿Quiero int, double, String, boolean...?



DataInputStream / DataOutputStream

¿Quiero guardar objetos Java?



ObjectInputStream / ObjectOutputStream

¿Hay recursos que cerrar?



try-with-resources

Y para casi todo lo anterior:

```
try {  
    // trabajar con el fichero  
} catch (IOException e) {  
    // gestionar el error  
}
```

Consejo para los alumnos: si se aprenden bien `File`, `BufferedReader`, `BufferedWriter`, `Properties`, `DataInputStream`, `DataOutputStream`, `IOException` y `try-with-resources`, ya tienen una base muy sólida para la programación de ficheros en Java.